ConectaStore

Sistema de Recomendação de Produtos Baseado em Grafos

Relatório técnico - Data Structure Strategy and Implementation

Aluno	Thiago Alves Antunes Pereira
Curso	ADS/EAD

1. Análise do problema

A MegaStore tem um catálogo enorme, e a recomendação que existe hoje mostra os itens mais vendidos ou os da mesma categoria. Isso é igual para todo cliente e não leva em conta o que cada um já comprou, então ajuda pouco a descobrir algo novo. Quem perde é o cliente, que não encontra o que serviria para ele, e a empresa, que deixa passar venda cruzada, ticket médio e fidelização.

Os dados necessários já existem (produtos, categorias, avaliações, histórico de compras). O que falta é ligá-los. Uma recomendação boa quase sempre é um caminho: o cliente comprou um produto, outro cliente comprou o mesmo produto e esse outro cliente também levou um terceiro. Guardar isso em tabelas soltas é ruim, e guardar como grafo é natural, por isso o projeto foi modelado assim.

2. Modelagem como grafo

2.1 Vértices

Há três tipos de vértice, todos no enum `NodeData`.

Vértice	Dados	Para que serve
Cliente	nome	ponto de partida da recomendação
Produto	nome, categoria, preço, rating	é o que se recomenda
Categoria	nome	agrupa produtos

2.2 Arestas e pesos

As arestas têm um tipo (`EdgeType`) e um peso. Quando a relação vale nos dois sentidos, guardo duas arestas dirigidas, uma em cada direção. Assim dá para andar nos dois sentidos sem varrer o grafo inteiro atrás de quem aponta para um vértice.

Aresta	De -> Para	Peso	Criada quando
Purchased	cliente -> produto	1,0	registra uma compra
BoughtBy	produto -> cliente	1,0	registra uma compra (inversa)
BelongsTo	produto -> categoria	1,0	cadastra o produto
Contains	categoria -> produto	1,0	cadastra o produto (inversa)
SimilarTo	produto <-> produto	0,0 a 1,0	define similaridade (nos dois sentidos)

Só a `SimilarTo` usa o peso de verdade, porque é a única relação com intensidade variável. Nas outras o peso é 1,0 e a importância de cada uma entra na pontuação do algoritmo (seção 6).

2.3 Tipo de grafo

O grafo é dirigido, ponderado e esparso, com vários tipos de vértice e de aresta. Clientes e produtos formam algo parecido com um grafo bipartido, e as categorias entram como um terceiro grupo que agrupa produtos.

3. Lista de adjacência e matriz de adjacência

Considerando V vértices e E arestas:

	Lista de adjacência	Matriz de adjacência
Espaço	O(V + E)	$O(V^2)$
Vizinhos de um vértice	O(grau)	O(V)
Existe a aresta (u, v)?	O(grau de u)	O(1)
Adicionar aresta	O(1)	O(1)
Adicionar vértice	O(1)	$O(V^2)$ para redimensionar

No maior cenário do benchmark (40 categorias, 50.000 produtos, 5.000 clientes) o grafo tem uns 55.000 vértices e em torno de 300.000 arestas: 2 por produto (categoria), 2 por compra e 2 por par de similares. Uma matriz teria 55.000^2, cerca de 3 bilhões de células. Mesmo com 1 byte por célula dá uns 3 GB para guardar algo com 0,01% de preenchimento. Num catálogo de milhões de produtos nem caberia na memória.

A lista gasta memória só com o que existe e entrega o que o algoritmo mais faz, que é percorrer os vizinhos de um vértice. A matriz só ganha em saber se uma aresta existe, e eu uso isso apenas em `registrar_compra`, para barrar compra repetida, num custo que é o número de compras do cliente. Por isso fiquei com a lista.

4. Estruturas de dados escolhidas

Estrutura	Uso	Motivo
HashMap<NodeId, Vec<Edge>>	lista de adjacência	vizinhos de qualquer vértice em O(1) médio, sem exigir IDs contíguos
Vec<Edge>	vizinhos de um vértice	memória contígua, percurso rápido e inserção no fim barata
HashMap<NodeId, NodeData>	dados dos vértices	consulta por ID em O(1) médio
HashMap<String, NodeId>	índice por nome	achar por nome sem varrer o grafo
HashMap<NodeId, f64>	scores	acumula pontos e deixa uma só entrada por produto
HashSet<NodeId>	visitados e já comprados	pertencimento em O(1); evita ciclos e filtra o que o cliente tem
VecDeque	fila da BFS	entrada e saída em O(1); num Vec tirar do início custa O(n)

Como os IDs dos vértices são sequenciais, um `Vec` indexado pelo ID seria um pouco mais compacto e rápido que o `HashMap` de vértices. Mantive o `HashMap` para não amarrar o código a esse formato de ID (se os dados vierem de um banco, por exemplo) e porque nos volumes testados a diferença não deve aparecer.

5. Percursos: BFS e DFS

As duas buscas têm limite de profundidade e um conjunto de visitados, então os ciclos que o grafo naturalmente tem (Purchased e BoughtBy apontam um para o outro) não repetem nós nem travam.

No pior caso custam O(V + E), mas com o limite só passam pela vizinhança que interessa.

A BFS usa fila e visita por camadas, então a distância que ela acha é a menor. Isso importa na recomendação por produto, onde um similar direto vale mais que um similar de similar. Fiz também uma variante, `bfs_tipo`, que só atravessa arestas de um tipo, usada para andar só por `SimilarTo`.

A DFS é recursiva e fica disponível no menu para explorar o grafo. Não a uso para ranquear porque ela não dá a menor distância. O limite de profundidade também evita estouro de pilha.

6. Algoritmo de recomendação

6.1 A partir de um cliente

Para cada produto P que o cliente C já comprou, o algoritmo segue três caminhos e dá pontos aos produtos que encontra. O que C já tem fica num conjunto e nunca vira candidato.

Caminho	Ideia	Pontos
C -> P -> outro cliente -> outro produto	quem comprou o mesmo também levou isto	+1,0 por comprador em comum
C -> P -> categoria -> outro produto	mesma categoria de algo que já comprou	+0,5
C -> P -> produto similar	similaridade cadastrada	peso da aresta

Os pontos vão para um `HashMap` de produto para score e se acumulam: um produto que apareceu por colaboração e por categoria vale mais que um que apareceu só pela categoria. No fim os candidatos são ordenados do maior para o menor score (empate resolvido pelo ID, para a saída não mudar entre execuções) e a lista é cortada em N.

Os pesos 1,0, 0,5 e o da similaridade são constantes no código. Escolhi assim porque o que outras pessoas compraram me parece um sinal mais forte que a categoria. Com dados reais daria para calibrar.

6.2 Por que a recomendação por cliente não usa a BFS pronta

No fundo ela é uma busca em largura de profundidade 3, só que separada por tipo de aresta. A BFS comum marca cada vértice como visitado uma única vez, e aqui eu preciso contar quantos caminhos chegam a cada candidato: dois compradores em comum valem mais que um. Por isso essa parte usa laços aninhados com o mapa de scores acumulando os pontos. A BFS de verdade entra na recomendação por produto, onde o que interessa é a menor distância.

6.3 A partir de um produto

Duas partes. Uma BFS de profundidade 2 só pelas arestas de similaridade dá 1/distância pontos a cada produto alcançado (similar direto 1,0, similar de similar 0,5). Depois, cada cliente que comprou o produto contribui com +1,0 para os outros itens que comprou. O próprio produto nunca entra na lista.

6.4 Duplicatas

Tratei em dois lugares. Na lista final, cada produto tem uma só entrada no mapa de scores. Nos dados, `registrar_compra` confere se a aresta já existe e, se existir, não faz nada; sem isso, uma

compra repetida criaria arestas repetidas e inflaria o score. As duas coisas têm teste.

7. Complexidade de tempo e espaço

V é o número de vértices e E o de arestas. Na recomendação por cliente, k é o número de produtos que o cliente comprou, b a média de compradores por produto, p a média de compras por cliente, m o tamanho médio das categorias, s a média de similares por produto e c o número de candidatos distintos.

Operação	Tempo	Comentário
Cadastrar categoria, cliente ou produto	O(1) médio	inserção em HashMap; o produto cria 2 arestas
Consultar por ID ou nome	O(1) médio	HashMap
Registrar compra	O(k)	confere a duplicata percorrendo as compras do cliente, depois 2 inserções
Definir similaridade	O(1)	2 arestas
Listar produtos, categorias ou clientes	O(V)	varre os vértices
BFS ou DFS com limite	O(V + E) no pior caso	na prática só a vizinhança
Recomendar por cliente	O(k(bp + m + s) + c log c)	gerar candidatos e depois ordenar
Recomendar por produto	O(V + E) no pior caso; em geral O(bp + s)	BFS limitada e compradores do produto

O custo de recomendar não depende do tamanho total do grafo, e sim da vizinhança do cliente. Dobrar o catálogo não dobra o tempo, a menos que as categorias cresçam (m) ou os produtos ganhem mais compradores (b).

No espaço, o grafo ocupa O(V + E). Cada aresta tem destino, tipo e peso e ocupa 24 bytes com o alinhamento, então 300.000 arestas ficam em uns 7 MB, sem contar a sobrecarga dos `Vec` e `HashMap`. Cada recomendação usa mais O(c) para os scores e O(k) para o conjunto de comprados, e a BFS usa O(V) no pior caso.

8. Desempenho e escalabilidade

8.1 Como medi

O gerador (`bench.rs`) usa um xorshift simples com seed fixa, então os dados são sempre os mesmos. Os produtos entram nas categorias em rodízio, os da mesma categoria são ligados em cadeia por similaridade 0,3 e cada cliente compra produtos sorteados. Com `Instant`, em modo release (opção 9 do menu), medi o tempo de gerar o grafo e o de uma recomendação top 10 para o primeiro cliente.

8.2 Resultados

Categorias	Produtos	Clientes	Compras/cli.	Nós	Geração	Recomendação
10	100	50	5	a preencher	a preencher	a preencher

Categorias	Produtos	Clientes	Compras/cli.	Nós	Geração	Recomendação
20	1.000	300	8	a preencher	a preencher	a preencher
30	10.000	2.000	10	a preencher	a preencher	a preencher
40	50.000	5.000	10	a preencher	a preencher	a preencher

Máquina: [processador, memória, sistema operacional].

8.3 Escalabilidade

A geração deve crescer de forma linear com vértices e arestas, já que é basicamente inserção em HashMap. A recomendação deve crescer bem mais devagar, porque depende da vizinhança e não do total.

O gargalo que eu espero é a expansão por categoria. No maior cenário cada categoria tem cerca de 1.250 produtos (50.000 divididos por 40), e cada produto comprado joga até 1.250 candidatos no mapa de scores. Com poucas categorias muito grandes, isso passa a dominar o tempo. Uma saída seria limitar quantos produtos de cada categoria entram (os mais bem avaliados, por exemplo) ou usar subcategorias menores.

Se o sistema precisasse crescer mais, eu faria quatro coisas: calcular antes as recomendações dos clientes mais ativos e atualizar em lote; trocar o `HashMap` de vértices por um `Vec`; usar um heap de tamanho N no lugar de ordenar todos os candidatos (O(c log N) em vez de O(c log c)); e limitar quantos compradores de cada produto são consultados.

9. Conclusão

Modelar o catálogo como grafo deixa recomendar com base no que os clientes realmente compraram, e não só no ranking de vendas. A lista de adjacência atende bem porque o grafo é esparso e o algoritmo vive de percorrer vizinhos, e os HashMap e HashSet em volta cuidam de consulta por ID, soma de scores sem duplicata e filtro do que já foi comprado. O custo de uma recomendação acompanha a vizinhança do cliente, não o tamanho do catálogo. O que eu vigiaria em escala é o tamanho das categorias.